CREATING BATCH FILES

How do you create a batch file?

Any text editor (an application that saves files as plain text files) will work to create batch files. The most popular application for creating batch files is the Microsoft Notepad utility included with Windows. Notepad has been included with every version of Windows since version 1.0 was released in 1985. (Which, I suspect, makes Notepad older than many of the readers of this book!) Notepad is very bare bones, but it will make a clean text file for you to use as a batch file. You'll need to make sure you save it as a BAT file instead of a TXT file - go to the File menu, and then to Save As, change the "Save As Type" to "All Files", and type the file name with a BAT extension.

It's a bad idea to use a full-featured word processor like Microsoft Office Word or LibreOffice Writer to create your batch file. The standard save formats of full-featured word processors are obviously not text files, and even when saving a Word document as a TXT file, it tends to be formatted incorrectly. If you want a more full-featured text editor, there are several free programs available - Notepad++ is one of the more popular ones.

Unfortunately, Windows does not include a text editor you can use from the Command Prompt. It's possible to create text files from the Command Prompt by using the ECHO command and output redirection, but even then, there's no way to edit the file once it has been created. This is one area where Mac OS X and numerous Linux distributions have the advantage over the Windows Command Prompt, since they include the vi text editor, which lets you edit text files from the command line (assuming you master vi's syntax and commands, of course).

BATCH COMMANDS

You can put practically any command in a batch file. There are, however, a set of ten commands that are particularly useful for adding functionality to batch files (and some of them only work properly when used in a batch file). Using them, you can create limited programs that accept user input, complete with assigned variables. These will be simple programs - to create really complex effects, you'd need to learn WMI (Windows Management Instrumentation) syntax or a programming language like C# or Visual Basic. Nevertheless, you can use these commands to create useful batch files.

The ten batch file commands are ECHO, CALL, FOR, PAUSE, CHOICE, GOTO, REM, IF, SHIFT, SET. Some of them are beyond the scope of this book, but we'll take a look at the simpler ones in this chapter.

ECHO

The ECHO command does just what its name indicates - it "echoes" back a line of text. Entered from the command line, the command will "echo" back whatever text follows the command. For instance, this command would generate an output of "HELLO!"

ECHO HELLO!

The ECHO command has two uses in creating batch files. First, you can use it to display directions on the screen, or an explanation of what the batch file is doing. Second, you can use it to generate cleaner output from your batch files. By default, the batch file displays every command on the screen, along with its

output. However, prefixing any command with a @ symbol prevents it from appearing on the screen. And if you use the @ symbol with the ECHO OFF command, it makes the prompt disappear for the duration of the batch file:

@ECHO OFF

This is useful if you don't want your batch file to clutter up the screen while it runs.

PAUSE

The PAUSE command does exactly what its name says. It pauses processing of the batch file until you press a key to continue. Issuing the PAUSE command from the Command Prompt generates this output:

Press any key to continue...

The Command Prompt will wait until you press a key to return control to the prompt. When inserted into a batch file, PAUSE stops the execution of the file, which is useful when you want to view some output before it scrolls off the top of the screen.

REM

The REM command doesn't actually do anything.

Despite that, it is quite useful. REM stands for "remark", and when used in a batch file, the Command Prompt will ignore any text that comes after it. This is handy for inserting "remarks", or notes, into your batch file. Why would you want to do that? The REM command allows you to document your batch files. If you write a batch file and need to edit it a year later (because, say, some server names or IP addresses changed) you might look at some of the commands and wonder why you put them in there. REM lets you make notes as you write the batch file, so you can examine them later and remember why you put the file together the way that you did.

The second use of the REM command is for testing. Using REM, you can "comment out" a command by prefixing REM before it. The Command Prompt will then treat the command as a remark and ignore it. This comes in handy when you are testing a batch file that does not work properly, allowing you to pin down which part of it isn't working as you thought.

GOTO

The GOTO command tells the Command Prompt to jump to a new location in the batch file. If you programmed old BASIC or Microsoft QBASIC programs, you might remember using GOTO statements with the numbered lines of code. GOTO in the modern Command Prompt works a bit different. GOTO jumps to a text line designated by a colon character (:) and resumes processing the commands in order. For instance, consider a GOTO command that looks like this in your batch file:

GOTO :SELECT

If you have a command like this, GOTO will jump to the line in the batch file beginning with :SELECT.

CHOICE

The CHOICE command does exactly what its name implies - it presents you with a choice. Typing the CHOICE command at the prompt generates an output that looks like this:

[Y, N]?

This looks rather similar to other Yes or No choices offered by other Command Prompt commands. By itself, CHOICE does nothing. You need to use it in conjunction with other commands (particularly the SET command) in order to make it useful. However, you can alter the appearance of the CHOICE command with a few convenient command switches. The /M switch lets you add a custom message to the CHOICE command (with the message itself in quotation marks). This command is generates a message of "Hello! Press Y or N":

CHOICE /M "Hello! Press Y or N"

If you want to have different options than to press Y or N, you can modify the options with the /C switch. This command prompts the user to press 1, 2, or 3, rather than Y or N:

CHOICE /C 123

You can also use the CHOICE command with a default option, and set it to choose that default option after a timeout measured in seconds. This version of the CHOICE command sets the default option to Y, after waiting 90 seconds for user input:

CHOICE /D Y /T 90

Using these switches, you can customize the choice offered by CHOICE to fit your needs.

SET

The SET command is a powerful tool that ties together the GOTO and CHOICE commands, allowing you to offer real choices in your batch files. By itself, the SET command primarily works with "environment variables." Environment variables are a number of system variables with preset values that control the way both the Command Prompt and Windows itself work. To see the environment variables for your system, use the SET command from the prompt without any switches:

SET

This generates quite a long list of output - Windows uses numerous environment variables. To quickly find the value for any one variable, you can use the ECHO command. However, you need to use the ECHO command with special characters. The TIME variable displays the system time, so you might be

tempted to use this command:

ECHO TIME

However, this will only display the word "TIME" on the screen. To use ECHO to display system variables, you'll need to enclose the variable within a pair of percentage characters (%), like this:

ECHO %TIME%

This time, rather than just spitting out "TIME", the output will look like this:

22:13:30.56

You can also use the SET variable to create your own environment variables. To create a variable entitled "TODAY" with a value of "Wednesday", use this command:

SET TODAY=Wednesday

Then if you use that variable with ECHO, it will return with a value of "Wednesday":

ECHO %TODAY%

Assign your own variables might seem like a pointless game, but it becomes useful when working with your own batch files. Using the SET command, you can create a batch file that will execute instructions based on the user's choices. With the /P switch, you can instruct SET to create a new variable based upon the user's input - in effect, the /P switch combines CHOICE and SET into a single command. Let's say you wanted to create a variable named INPUT, and offer the user a choice between Option 1 and Option 2. The SET command to create such a choice would look something like this:

SET /P INPUT="Press 1 for Option 1, and Press 2 for Option 2: 1,2: "

Combined with the GOTO and the IF commands, this lets you create batch files that offer choices. (Note that the message text has to be in quotation marks, and the actual option keys themselves need to be enclosed in colon characters, as shown in the example.)

IF

The IF command ties together batch processing. The IF command responds to the variable created by the SET command. For every potential value of the previously created variable, you can then tell the IF command to perform a specific task. Generally, the best use of the IF command is to issue the GOTO command, which will then route Command Prompt to the section of the batch file containing the necessary commands.

This example shows the best use of the commands we've explored in this

chapter. This batch file offers the user a choice between pressing "C" for a listing of the root directory of the C drive and "D" for a listing of the root directory of the D drive. Depending upon which choice the user takes, he is moved to a different portion of the batch file, which then executes the necessary commands:

```
@echo off

set /p input="Press C for C:\, and press D for D: C,D: "

if %input%==1 goto C

if %input%==2 goto D

:C

dir C:\

goto end

:D

dir D:\
```

goto end

:end

By following this example, you can create your own batch files using variables and choices.

***

Chapter 14 – Task Management

If you've used Windows for any length of time, then you've probably encountered the Task Manager – the graphical utility that lets you observe system performance, see which applications and processes are running on your system, and manually kill processes that become unreliable or freeze up. There are several commands that let you perform many of the Task Manager's functions from the command line. In this chapter, we'll show you how to manage processes from the command prompt. Using the commands in this chapter, you'll be able to list running processes on your computer, and terminate ones that become troublesome.

WHAT IS A PROCESS?

In Windows, a “process” doesn't refer to the specific procedure you use to do something, or a standardized method for carrying out a task. Rather, a process is defined as the active instance of a computer program. For example, if you ran the Firefox web browser, Firefox would be a running process. Once you exit Firefox and close your browsing session, the process would exit and disappear.

A process can also be a background program running on your computer, either a background program you installed yourself or a component of Windows. If you launch Task Manager (or use the commands described later in this chapter) you will see dozens of processes. Most of them will be background components of Windows itself, like the print spooler or the networking services.

Generally, Windows handles its processes, whether applications or built-in Windows services, pretty well, and you don’t need to think about them. But like

anything else, there can be problems. Sometimes a process will freeze up and refuse to quit, and you will need to forcibly terminate it. Additionally, some processes can seize so much of your computer's available memory or CPU power that your system slows to a crawl and becomes nonresponsive. If that happens, you'll need to find out which process is hogging the resources and terminate it.

In the next section, we'll show you how to list currently running processes on your computer.

LISTING RUNNING PROCESSES

To view the running processes on your computer, use this command:

TASKLIST

This will immediately list every running process on your Windows computer. This will probably scroll off the screen, so you might want to pipe the output to the more command to let you view the output one screen at a time:

TASKLIST | MORE

By default, tasklist lists five different pieces of information about the processes running on your system. The first, Image Name, is basically a friendly name that allows you to find the process quickly in a list. The second, PID, stands for Process Identifier. Windows assigns a Process Identifier number to every process

on your system, and the PID turns up in several log files.

The third piece of information, Session Name, identifies whether or not a process is a Services session or a Console session. Basically, a Services process is one launched by Windows to run in the background, while a Console process was launched by the user logged into the computer. On a Windows Server system, you can also see RDP under Session Name – that means a user logged onto the server via Remote Desktop Services launched the process.

The fourth piece of information is Session Number, listed as Session#. On a client Windows system, you'll generally only see two numbers – 0 for processes launched by Windows, and 2 for processes launched by the logged-in user. On a Windows Server system running Remote Desktop, you will often see more numbers.

The final piece of information, Mem Usage, simply lists the amount of memory a specific process is using.

You can get even more information from the tasklist command if you use it with the /v option:

TASKLIST /V

When used with the /v command, TASKLIST provides four additional columns of information. The first, Status, displays what the process is currently doing. Most of the time, this will say Running, but if a process has frozen up, it could say Not Responding. Processes launched by Windows will say Unknown, since Windows has control of them.

The second piece of information is User Name, which lists the user that launched the process. Usually, this will be the user currently logged into the system. If you are using this command on a Remote Desktop Services system, this can help you track down which user launched which process, which is very useful if a specific process is hogging all the system resources.

The third piece of additional information is CPU Time, which shows how much CPU time each process has used. As with Mem Usage, this can help you track down a process that is using too many system resources. The final piece of information is Window Title. If a process has also opened a window on the desktop, the title of the window is listed here. If you have an errant application that refuses to close and you do not know the name of its process, you can help use the title of its window to find its process here.

TERMINATING PROCESSES

Generally, when processes work, you don't have to think about them. If you are going to the trouble of listing processes, that probably means one or more processes have malfunctioned and you need to forcibly shut them down. Listing processes allows you to find which processes you need to terminate. To terminate a process from the Command Prompt, use this command:

TASKKILL

By itself, taskkill only spits on an error message. To make proper use of it, you need to specify either the image name or the PID of the process you wish to terminate. For instance, if you wanted to terminate a process with an image name of application.exe, you would use TASKKILL with the /IM switch to

specify the image name:

TASKKILL /IM APPLICATION.EXE

This would kill the APPLICATION.EXE process immediately.

You can also use the /PID switch to specify TASKKILL to terminate a process with a specific PID. To return to our previous example, if APPLICATION.EXE has a PID of 1234, you can use this command to terminate it by PID:

TASKKILL /PID 1234

Additionally, the TASKKILL command offers two additional switches you can use with either the /PID switch or the /IM switch. First, the /F switch forcibly terminates the process, which is useful if you have a recalcitrant process that simply refuses to terminate. Second, the /T switch also terminates any child processes that the process launch. So, if you wanted to terminate a process with a PID of 1954, while also forcing it to terminate and terminating any child processes, the command would look like this:

TASKKILL /T /F /PID 1954

The process would then be terminated, along with any child processes.

***

Afterward

I hope this book has been a useful introduction to the Windows Command Prompt.

Windows - and the entire technology world - changes very rapidly. Since this is an ebook, I hope to update it regularly to reflect the changes made to Windows in particular and computing technology in general.

***

Other books by the author

The Demonsouled Saga

MAZAEL CRAVENLOCK is a wandering knight, fearless in battle and masterful with a sword.

Yet he has a dark secret. He is Demonsouled, the son of the ancient and cruel Old Demon, and his tainted blood grants him superhuman strength and speed. Yet with the power comes terrible, inhuman rage, and Mazael must struggle to keep the fury from devouring him.

But he dare not turn aside from the strength of his blood, for he will need it to face terrible foes.

The priests of the San-keth plot and scheme in the shadows, pulling lords and kingdoms upon their strings. The serpent priests desire to overthrow the realms of men and enslave humanity. Unless Mazael stops them, they shall force all nations to bow before the serpent god.

The Malrag hordes are coming, vast armies of terrible, inhuman beasts, filled with a lust for cruelty and torment. The Malrags care nothing for conquest or treasure, only slaughter. And the human realms are ripe for the harvest. Only a warrior of Mazael’s power can hope to defeat them.

The Dominiar Order and the Justiciar Order were once noble and respected, dedicated to fighting the powers of dark magic. Now they are corrupt and cynical, and scheme only for power and glory. They will kill anyone who stands in their way.

To defeat these foes, Mazael will need all the strength of his Demonsouled blood.

Yet he faces a far more terrible foe.

For centuries the Old Demon has manipulated kings and lords. Now he shall seize the power of the Demonsouled for himself, and become the a god of torment and tyranny.

Unless Mazael can stop him.

Read Demonsouled (http://www.jonathanmoeller.com/writer/?page_id=880) for free. Mazael's adventures continue in Soul of Tyrants (http://www.jonathanmoeller.com/writer/?page_id=911), Soul of Serpents (http://www.jonathanmoeller.com/writer/?page_id=1287), Soul of Dragons (http://www.jonathanmoeller.com/writer/?page_id=1727), Soul of Sorcery (http://www.jonathanmoeller.com/writer/?page_id=1845), Soul of Skulls (http://www.jonathanmoeller.com/writer/?page_id=2808), and Soul of Swords (http://www.jonathanmoeller.com/writer/?page_id=3599), along with the short stories The Wandering Knight (http://www.jonathanmoeller.com/writer/?page_id=3073), The Tournament Knight (http://www.jonathanmoeller.com/writer/?page_id=3677), and The Dragon's Shadow (http://www.jonathanmoeller.com/writer/?page_id=2635). Get the first three books bundled together in Demonsouled Omnibus One (http://www.jonathanmoeller.com/writer/?page_id=4442).

The Ghosts Series

Once CAINA AMALAS was the shy daughter of a minor nobleman, content to spend her days in her father's library.

Then sorcery and murder and her mother's treachery tore her life apart.

Now she is a nightfighter of the Ghosts, an elite agent of the spies and assassins of the Emperor of Nighmar. She is a master of disguise and infiltration, of stealth and the shadows.

And she will need all those skills to defend the Empire and stay alive.

Corrupt lords scheme and plot in the shadows, desiring to pull down the Emperor and rule the Empire for their own profit and glory. Slave traders lurk on the fringes of the Empire, ready to seize unwary commoners and sell them into servitude in distant lands. Yet both slave traders and cruel lords must beware the Ghosts.

The Magisterium, the Imperial brotherhood of sorcerers, believe themselves the rightful masters of the Empire. With their arcane sciences, they plan to overthrow the Empire and enslave the commoners, ruling all of mankind for their own benefit. Only the Ghosts stand in the path of their sinister plans.

And the Moroaica, the ancient sorceress of legend and terror, waits in the

shadows, preparing to launch a war upon the gods themselves. She will make the gods pay for the suffering of mankind...even if she must destroy the world to do it.

Caina Amalas of the Ghosts opposes these mighty enemies, but the cost might be more than she can bear.

Read Child of the Ghosts (http://www.jonathanmoeller.com/writer/?page_id=1057) for free. Caina's adventures continue in Ghost in the Flames (http://www.jonathanmoeller.com/writer/?page_id=1265), Ghost in the Blood (http://www.jonathanmoeller.com/writer/?page_id=1383), Ghost in the Storm (http://www.jonathanmoeller.com/writer/?page_id=1931), Ghost in the Stone (http://www.jonathanmoeller.com/writer/?page_id=2520), Ghost in the Forge (http://www.jonathanmoeller.com/writer/?page_id=3181), Ghost in the Ashes (http://www.jonathanmoeller.com/writer/?page_id=3932), Ghost in the Mask (http://www.jonathanmoeller.com/writer/?page_id=4231), and Ghost in the Surge (http://www.jonathanmoeller.com/writer/?page_id=4637), along with the short stories Ghost Aria (http://www.jonathanmoeller.com/writer/?page_id=3243), Ghost Claws (http://www.jonathanmoeller.com/writer/?page_id=3930), Ghost Omens (http://www.jonathanmoeller.com/writer/?page_id=4235), The Fall of Kyrace (http://www.jonathanmoeller.com/writer/?page_id=4258), Ghost Thorns (http://www.jonathanmoeller.com/writer/?page_id=4639), Ghost Undying (http://www.jonathanmoeller.com/writer/?page_id=4662), Ghost Light (http://www.jonathanmoeller.com/writer/?page_id=5653), and Ghost Dagger (http://www.jonathanmoeller.com/writer/?page_id=2371), and the prequel novels Blade of the Ghosts (http://www.jonathanmoeller.com/writer/?page_id=6220) and Champion of the Ghosts (http://www.jonathanmoeller.com/writer/?page_id=6304). Get the first three books bundled together in The Ghosts Omnibus One (http://www.jonathanmoeller.com/writer/?page_id=4484).

The Ghost Exile Series